Authentication

Validating Requests

An overview on validating requests originating from HubSpot to your backend service.

To ensure that the requests that your integration is receiving from HubSpot are actually coming from HubSpot, several headers are populated in the request. You can use these headers, along with fields of the incoming request, to verify the signature of the request.

The method used to verify the signature depends on the version of the signature:

> To validate a request using the latest version of the HubSpot signature, use the `X-HubSpot-Signature-V3` header and follow the **associated instructions for validating the v3 version of the signature**.
>
> For backwards compatibility, requests from HubSpot also include older versions of the signature. To validate an older version of the signature, check the `X-HubSpot-Signature-Version` header, then follow the associated instructions below based on whether the version is **v2** or **v1**.

In the instructions below, learn how to derive a hash value from your app's client secret and the fields of an incoming request. Once you compute the hash value, compare it to the signature. If the two are equal, then the request has passed validation. Otherwise, the request may have been tampered with in transit or someone may be spoofing requests to your endpoint.

If you're **building an app** with OAuth authentication, check out the blog post below for guidance on how to validate requests from HubSpot:

Blog Post: Production-Ready OAuth Token Management

Read more on the HubSpot Developer Blog ›

HubSpot Developers

›

`hubspot-Request-Timestamp` header.

When validating a request using the X-HubSpot-Signature-v3 header, you'll need to

Reject the request if the timestamp is older than 5 minutes.

In the request URI, decode any of the URL-encoded characters listed in the table below. You do not need to decode the question mark that denotes the beginning of the query string.

Encoded value	Decoded value
%3A	:
%2F	/
%3F	?
%40	@
%21	!
%24	$
%27	'
%28	(
%29	)
%2A	*
%2C	,
%3B	;

Create a utf-8 encoded string that concatenates together the following: `requestMethod` + `requestUri` + `requestBody` + timestamp. The timestamp is provided by the `X-HubSpot-Request-Timestamp` header.

Create an HMAC SHA-256 hash of the resulting string using the application secret as the secret for the HMAC SHA-256 function.

Base64 encode the result of the HMAC function.

The code snippets in the section below detail how you could incorporate v3 request validation for a `POST` request if you were running a backend service to handle incoming requests.

> ⚠ Keep in mind that the code blocks below omit certain dependencies you might need to run a fully-featured backend service. Confirm that you're running the latest stable and secure libraries when implementing request validation for your specific service.

v3 request signature examples

The code blocks in the tabs below provide examples of validating the v2 request signature using Node.js or Java.

Node.js Java PHP

HubSpot Developers

```
const bodyParser = require("body-parser");
const crypto = require("crypto");
const app = express();
const port = process.env.PORT || 4000;

app.use(bodyParser.urlencoded({ extended: false }));
app.use(bodyParser.json());

app.post("/webhook-test", (request, response) => {
  response.status(200).send("Received webhook subscription trigger");

  const { url, method, body, headers, hostname } = request;

  // Parse headers needed to validate signature
  const signatureHeader = headers["x-hubspot-signature-v3"];
  const timestampHeader = headers["x-hubspot-request-timestamp"];

  // Validate timestamp
  const MAX_ALLOWED_TIMESTAMP = 300000; // 5 minutes in milliseconds
  const currentTime = Date.now();
  if (currentTime - timestampHeader > MAX_ALLOWED_TIMESTAMP) {
    console.log("Timestamp is invalid, reject request");
    // Add any rejection logic here
  }

  // Concatenate request method, URI, body, and header timestamp
  const uri = `https://${hostname}${url}`;
  const rawString = `${method}${uri}${JSON.stringify(body)}${timestampHeader}`;

  // Create HMAC SHA-256 hash from resulting string above, then base64-encode i
  const hashedString = crypto.createHmac("sha256", process.env.CLIENT_SECRET).u

  // Validate signature: compare computed signature vs. signature in header
  if (crypto.timingSafeEqual(Buffer.from(hashedString), Buffer.from(signatureHe
    console.log("Signature matches! Request is valid.");
    // Proceed with any request processing as needed.
  } else {
    console.log("Signature does not match: request is invalid");
    // Add any rejection logic here.
```

HubSpot Developers

›

Validate requests using the v2 request signature

If your app is handling data from a **webhook action in a workflow**, or if you're returning data for an **app card**, the request from HubSpot is sent with the `X-HubSpot-Signature-Version` header set to `v2`. The `X-HubSpot-Signature` header will be an SHA-256 hash built using the client secret of your app combined with details of the request.

To verify this signature, perform the following steps:

- Create a string that concatenates together the following: `Client secret` + `http method` + `URI` + `request body` (if present)
- Create a SHA-256 hash of the resulting string.
- Compare the hash value to the signature.
  - If they're equal then this request has passed validation.
  - If these values do not match, then this request may have been tampered with in-transit or someone may be spoofing requests to your endpoint.

> **Please note:**
>
> The URI used to build the source string must exactly match the original request, including the protocol. If you're having trouble validating the signature, ensure that any query parameters are in the exact same order they were listed in the original request.
>
> The source string should be UTF-8 encoded before calculating the SHA-256 hash.

Example for a GET request

For a `GET` request, you'd need your app's client secret and specific fields from the metadata of your request. These fields are listed below with placeholder values included:

- Client secret: `yyyyyyyy-yyyy-yyyy-yyyy-yyyyyyyyyyyy`
- HTTP method: `GET`
- URI: `https://www.example.com/webhook_uri`

HubSpot Developers

›

After calculating a SHA-256 hash of the concatenated string above, the resulting signature you'd expect to match to the one in the header would be: `eee2dddcc73c94d699f5e395f4b9d454a069a6855fbfa152e91e88823087200e`

Example for a POST request

For a `POST` request, you'd need your app's client secret, specific fields from the metadata of your request, and a string representation of the body of the request (e.g., using `JSON.stringify(request.body)` for a Node.js service). These fields are listed below with placeholder values included:

Client secret: `yyyyyyyy-yyyy-yyyy-yyyy-yyyyyyyyyyyy`

HTTP method: `POST`

URI: `https://www.example.com/webhook_uri`

Request body: `{"example_field":"example_value"}`

The resulting concatenated string would be: `yyyyyyyy-yyyy-yyyy-yyyy-yyyyyyyyyyyyPOSThttps://www.example.com/webhook_uri{"example_field":"example_value"}`

After calculating a SHA-256 hash of the concatenated string above, the resulting signature you'd expect to match to the one in the header would be: `9569219f8ba981ffa6f6f16aa0f48637d35d728c7e4d93d0d52efaa512af7900`

After [SHA-ing] the signature, you could then compare the resulting expected signature to the one provided in the x-hubspot-signature header of the request:

> ⚠ The code snippets below details how you could incorporate `v2` request validation for a `GET` request if you were running a backend service to handle incoming requests. Keep in mind that the code block below is an example and omits certain dependencies you might need to run a fully-featured backend service. Confirm that you're running the latest stable and secure libraries when implementing request validation for your specific service.

v2 request signature examples

HubSpot Developers

```
// Introduce any dependencies. Only several dependencies re
const express = require("express");
const bodyParser = require("body-parser");
const crypto = require("crypto");
const app = express();

// Add any custom handling or setup code for your Node.js service here.
app.use(bodyParser.urlencoded({ extended: false }));
app.use(bodyParser.json());

// Example Node.js request validation code.
app.get("/example-service", (request, response, next) => {
  const { url, method, headers, hostname } = request;

  const requestSignature = headers["x-hubspot-signature"];

  // Compute expected signature
  const uri = `https://${hostname}${url}`;
  const encodedString = Buffer.from(`${process.env.CLIENT_SECRET}${method}${uri
  const expectedSignature = crypto.createHash("sha256").update(encodedString).d

  console.log("Expected signature: %s", requestSignature);
  console.log("Request signature: %s", expectedSignature);

  // Add your custom handling to compare request signature to expected signatur
  if (requestSignature !== expectedSignature) {
    console.log("Request of signature does NOT match!");
    response.status(400).send("Bad request");
  } else {
    console.log("Request of signature matches!");
    response.status(200).send();
  }
});
```

HubSpot Developers

›

header will be an SHA-256 hash built using the client secret of your app combined with details of the request.

To verify this version of the signature, perform the following steps:

- Create a string that concatenates together the following: `Client secret + request body` (if present).
- Create a SHA-256 hash of the resulting string.
- Compare the hash value to the value of the `X-HubSpot-Signature` header:
  - If they're equal then this request has passed validation.
  - If these values do not match, then this request may have been tampered with in-transit or someone may be spoofing requests to your endpoint.

Example for a request with a body:

```
// Client secret : yyyyyyyy-yyyy-yyyy-yyyy-yyyyyyyyyyyy
// Request body:
[
  {
    "eventId": 1,
    "subscriptionId": 12345,
    "portalId": 62515,
    "occurredAt": 1564113600000,
    "subscriptionType": "contact.creation",
    "attemptNumber": 0,
    "objectId": 123,
    "changeSource": "CRM",
    "changeFlag": "NEW",
    "appId": 54321
  }
]
```

v1 request signature examples

HubSpot Developers

```
NOTE: This is only an example for generating the expected ha
You will need to compare this expected hash with the actual hash in the
X-HubSpot-Signature header.

> const crypto = require('crypto')
undefined
> client_secret = 'yyyyyyyy-yyyy-yyyy-yyyy-yyyyyyyyyyyy'
'yyyyyyyy-yyyy-yyyy-yyyy-yyyyyyyyyyyy'
> request_body = '[{"eventId":1,"subscriptionId":12345,"portalId":62515,"occurr
'[{"eventId":1,"subscriptionId":12345,"portalId":62515,"occurredAt":15641136000
> source_string = client_secret + request_body
'yyyyyyyy-yyyy-yyyy-yyyy-yyyyyyyyyyyy[{"eventId":1,"subscriptionId":12345,"port
> hash = crypto.createHash('sha256').update(source_string).digest('hex')
'232db2615f3d666fe21a8ec971ac7b5402d33b9a925784df3ca654d05f4817de'
```

For each of the examples above, the resulting hash would be:

`232db2615f3d666fe21a8ec971ac7b5402d33b9a925784df3ca654d05f4817de`

Last modified on February 5, 2026

Was this page helpful? Yes No

Scopes
Previous

Create a new app using the CLI

HubSpot Developers

Copyright © 2026 HubSpot, Inc

Privacy Policy

Security

Website Accessibility

Manage Cookies